ARIana: The Intussusception Reduction Simulator

Instructor Manual

ARIana (Python/Tk)

A cross-platform simulator for teaching air-enema reduction of pediatric intussusception.

Disclaimer

This device is designed to help a trained pediatric radiologist teach a trainee the basics of reducing an intussusception with an air enema. It is intended to supplement rather than replace the experience of performing a supervised intussusception reduction on an actual patient. In other words, a trainee who has used this simulator a few times should not consider himself competent to perform an intussusception reduction in the absence of any further experience. There are nuances of the intussusception reduction procedure that are not within the scope of this device and which can only be gained through practical experience under the watchful eye of a trained pediatric radiologist.

Downloading:

Operating System Versions Supported:
ARIana currently supports Windows 7 or later and macOS 10.9 or later. There is also a Linux version available but it is currently in testing. If you are using an unsupported operating system, you are also welcome to compile for yourself using the Python files in the Source Code folder. Niutika is the recommended compiler. There are .yml files that include Niutika commands for Windows, Mac, and Linux, which can be used to compile your own version.

Windows:
Download *ARIana-Windows-Installer.zip*. Place the folder in a read/write folder and extract the zip file. **Do not remove the ARIana.exe file from the folder.** If you would like to put the app somewhere else, instead, create a shortcut by right clicking and pressing "Create a shortcut" and drag the created shortcut to your preferred location. Double click this shortcut to launch ARIana.exe. The first time you launch ARIana, it may say "Windows Protected Your PC. Microsoft Defender SmartScreen prevented an unrecognized app from starting. Running this app might put your PC at risk." This can be safely ignored–the error occurs because, since this app is open source and doesn't generate profit, it wasn't signed under a paid Microsoft license. All the code was written by Miles Fleisher and can be viewed in the Source Code folder and all dependencies have been thoroughly vetted. This issue can be easily fixed by clicking
the small "more info" link and then clicking "Run anyway." After doing this once, the warning should not show up again.

On Windows, it is important to make sure that you have the correct drivers and that they're up to date. This app is designed to work with an Extech HD 750 Differential Manometer, which uses a CP2102 USB to UART Bridge. The drivers for this bridge are available from Silicon Labs. If you are on Windows 10 (build 1803) or older (like Windows 9, 8, and 7), it's recommended to install the Windows VCP driver. If you're on Windows 10 (build 1803) or higher, it's recommended to install the Windows Universal Driver. If you are on Windows

10 but don't know if you're before or after version 1803, you can type *winver* into the Command Prompt and it will take you to a page listing the build number. If the build number is greater than or equal to 17134, then you are on a new version of Windows and should install the Universal driver here. If the build number is smaller than 17134, you are on an older build of Windows and should install the VCP driver here. To check if your driver is up to date, go to the *Device Manager* app, click view, verify "show hidden" is enabled, and expand "Ports (COM & LPT)." The driver should be called "Silicon Labs CP210x USB to UART Bridge (COMx)." Right click on the driver, click "Properties," and go into the Drivers tab. Note the Driver Version and Driver Date and compare them to the latest available on Silicon Labs' website. As long as your driver is in the same major version as the ones listed below (v11 for the universal driver or v6.7 for the VCP driver), it should probably be fine but, if you are having trouble connecting to the manometer, it might be worth updating. As of 9/3/25, the latest drivers are as shown below:

- *v11.4.0 → Current Universal driver (for Win10 1803+ / Win11).*
- *v6.7 (2020) → Latest legacy VCP driver (for Win7/8/early 10).*

The first time you run the app, you may experience a longer loading time than usual. This is because the app is generating multiple setting files. The app may also give you some alerts that say that the "checklist file could not be found." As long as the alert doesn't say that the "file couldn't be created," this is not an issue and can safely be ignored. On future startups, the app should load much faster.

Mac:
Download *ARIana-Mac-Installer.dmg*. Open the .dmg file and drag the ARIana app to your Applications folder. You may then safely eject the ARIana.dmg.

The first time you run the app, you may experience a longer loading time than usual. This is because the app is generating multiple setting files. The app may also give you some alerts that say that the "checklist file could not be found." As long as the alert doesn't say that the "file couldn't be created," this is not an issue and can safely be ignored. On future startups, the app should load much faster.

When launching the app for the first time, you may receive a warning that "the app cannot be scanned for malware." This is not an issue and it can be safely ignored. This warning is because the app was not signed with the Apple Developer License. This can be safely ignored–the error occurs because, since this app is open source and doesn't generate profit, it wasn't signed under a paid Microsoft license. All the code was written by Miles Fleisher and can be viewed in the Source Code folder and all dependencies have been thoroughly vetted. The warning can be bypassed by first going to *System Preferences>Security*, scrolling down to the bottom. Next, find the warning that says "ARIana was blocked to protect your Mac." Click *Open Anyway*, and give your admin password. After you've done this once, the app should run as expected in the future.

When you run the app for the first time, you will get a warning saying "Could not find 'preop_checklist.txt'. A default file has been created at: /Applications/ARIana.app/ Contents/Resources/ preop_checklist.txt

Edit this file to customize the Pre-Procedure Checklist." This warning can be safely ignored and only means that the app couldn't find the Pre-Procedure Checklist and is generating the default one in the proper location. This will not happen again after the initial setup.

Folder Layout:

The files in this GitHub are laid out as follows. <CaseID> is a placeholder for the names of different cases/patients.

Folder Layout
The project's folder structure is organized as follows:
Plain Text

```
repo/
├─ ARIana.py # Main application entry point for logic file(intussusception_trainer.py)
├─ Patients/
│  └─ <CaseID>/
│  ├─ <CaseID>_metadata.json # Case parameters (see schema below)
│  └─ Images/
│  ├─ Preprocedure/ # Pre-procedure images (e.g.,preprocedure_1.png)
│  ├─ Simulation/ # Simulation images (e.g.,simulation_1.png)
│  └─ Postprocedure/ # Post-procedure images (e.g.,postprocedure_1.png)
├─ ARIana_logo.png # Application icon/branding
└─ README.md
```

Adding your own preoperation checklist

Windows:
Go into the folder holding ARIana.exe, find *preop_checklist.txt, and* open this file in your text editor of choice, make changes to it, and save it. Make sure that you save the file as a .txt and don't duplicate the original. Note that some special characters may not be supported and can cause issues with the graphical interface. Reloading the app may be required for changes to take effect.

Mac:
Right click on the ARIana app and click "Show Package Contents." Navigate to *preop_checklist.txt* and open the file in your text editor of choice, make changes to it, and save it. Make sure that you save the file as a .txt and don't duplicate the original. Note that some special characters may not be supported and can cause issues with the graphical interface. Reloading the app may be required for changes to take effect.

Adding you own cases:

Windows:
Go into the folder holding ARIana.exe, find the folder named "Placeholder." Don't include spaces in the name of your folder. Copy this folder and rename it to the name of your patient/case. This is an extremely important step as the app will ignore folders named

"Placeholder." Enter this folder and rename the placeholder_metadata.json to <patient name>_metadata.json(the brackets should not be included; they are only there to emphasize the part of the name that should be replaced with the name of the case). Make sure that the name of the .json file and the name of the folder containing it match. Go into the *Images* folder and replace the images in each of the three folders (*Preoperation, Simulation,* and *Postoperation).* Make sure that you use the same naming convention as the placeholder images: <Foldername>_<imagenumber>.png. The image number dictates the order that images will be shown and, in the simulation folder, the order stage that an image will be associated with. If you have a perforation image, add it to the Images folder as the final image. You can also use any of the images from the included cases to create your new cases with new probabilities or features. Please make sure that all images are formatted as .png.

Now that you have renamed the folder, .json, and added your own images, it's time to edit the .json file. Open the .json file in any text editor. Make sure that the "*num stages"* value is set to the number of images in the Simulation folder, otherwise, some of the images may not display. The following is a list of values in the .json file and what they control.

Value	Control
name	The name displayed in the case selection screen
teaser	The short case description displayed in the case selection screen.
clinical_descrip	The longer description displayed when the "Take Vitals and History" is pressed in the preoperation screen
perf_data	This is the probability at each pressure listed that the patient will perforate. Pairs are listed as *[pressure, probability]* and are linearly interpolated for values in between the listed pressure/probability pairs listed
ret_data	This is the probability for retrogression (moving from the current stage to a previous stage). This uses the same linear interpolation logic as perf_data
coeff	This coefficient value effectively is the maximum probability of success when the pressure is at the maximum (180 mmHg). This value scales the success probabilities for all other pressures.
num_stages	The number of stages in the simulation. This should be set to the number of images in the *Simulation* folder
donstart	This value is *0* by default. If the patient has a

Value	Control
	contraindication and surgery is not advised, set it to *1.* Please note that this will cause the app to skip the simulation step entirely and display a warning if the user tries to start air enema reduction.

Hardware Setup:

While the software can be used with a virtual pressure slider and no external hardware, the full setup tends to be more immersive. The total materials required to build the setup are as follows:

- Computer running the ARIana software
- USB mini B cable that can plug into your computer
- Extech HD750 Differential Manometer
- ⅛" inner diameter flexible tubing
- ¼" inner diameter flexible tubing
- Insufflator with luer-lock output
- Luer lock to ¼" inner diameter flexible tubing adapter (can use a 3 way stopcock with one of the branches turned off and a luer lock to ¼" tubing adapter
- 150 mL bulb with tubing barb attachments for insufflator (¼" inner diameter) and manometer (⅛" inner diameter)
  - Optionally put this inside a modified doll
  - Can be replaced with a sealed syringe using epoxy or similar
- (optional) bleed valve with male ¼" inner diameter tubing on both ends to simulate slight leaks and inefficiencies
  - If you are using a bleed valve, make sure to connect the manometer onto or near the bulb instead of connecting it between the insufflator and bleed valve.

First

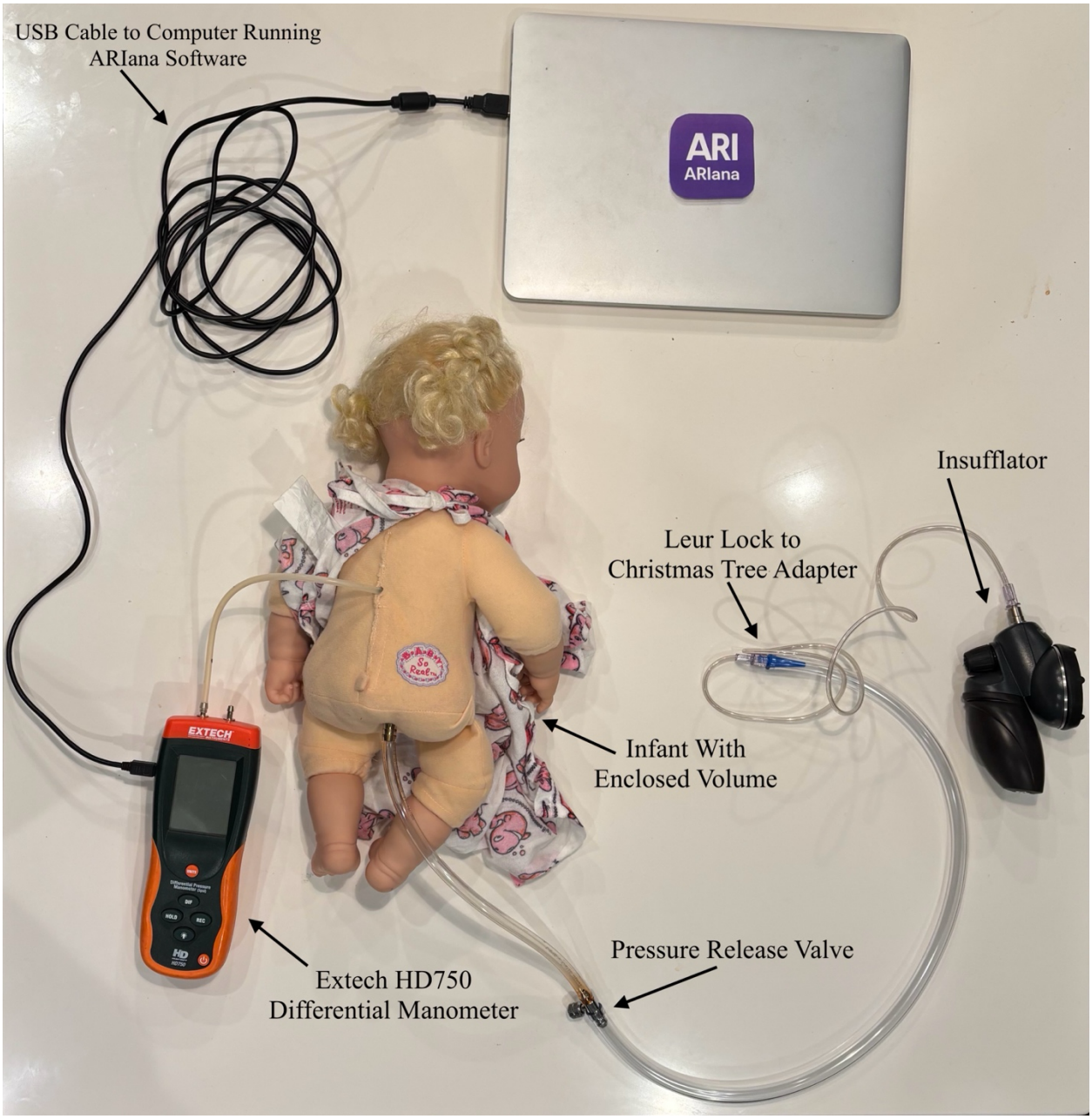

Figure 1: Full Setup Using Mannequin

Figure 1 shows the full setup with the bulb encased in the mannequin. The insufflator is attached to a leur lock to Christmas tree adapter. This Christmas tree adapter then goes to tubing with the appropriate diameter for our bleed valve. The bleed valve then connects to more tubing (in this case ¼" ID). This tubing connects to the ¼" barb on the mannequin. The smaller barb attaches to the manometer (Extech HD750 Differential Manometer) using 1/8" ID flexible tubing. Finally, the manometer is connected to the computer using the included USB cable.

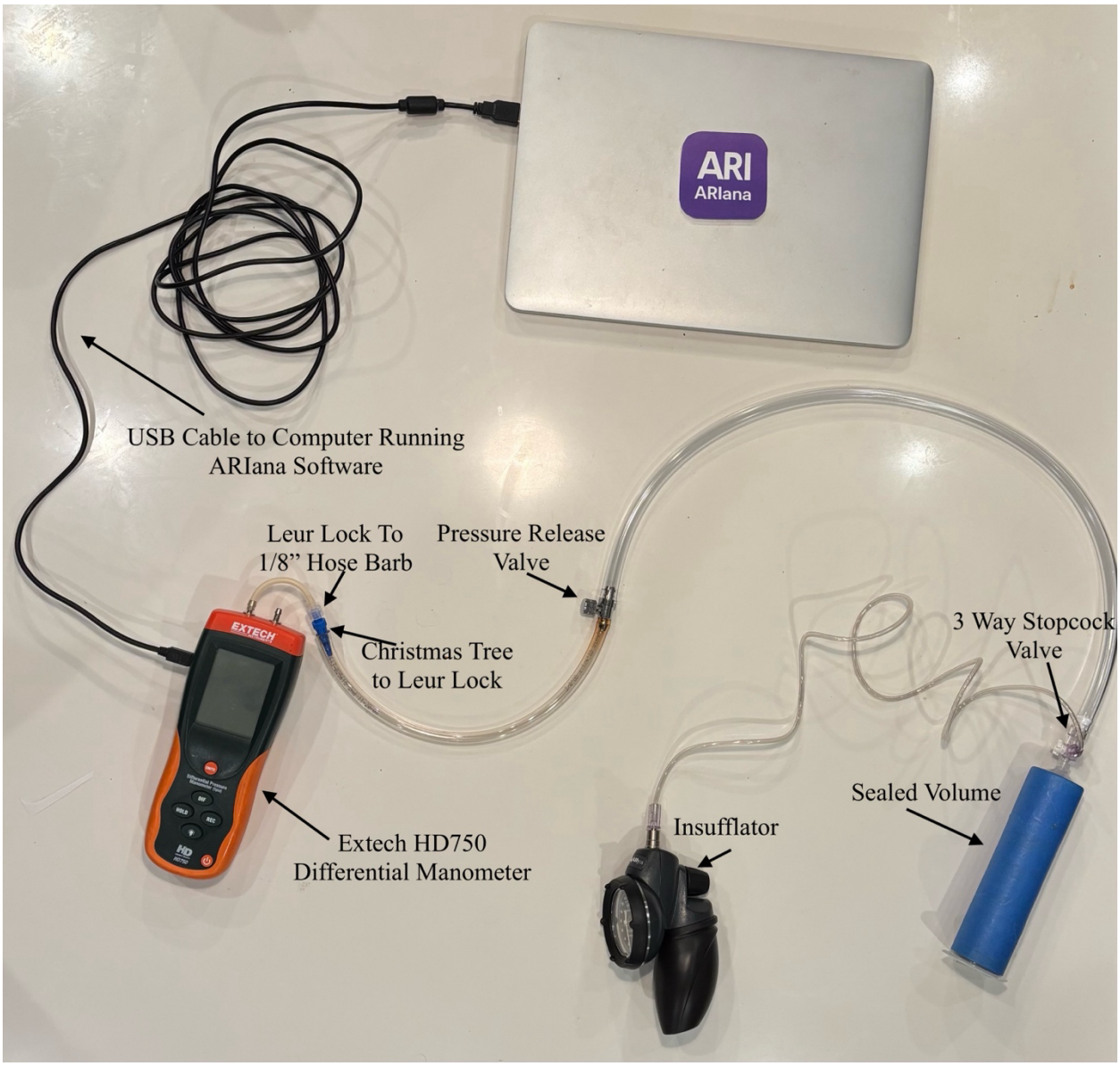

Figure 2: Simplified Setup without Mannequin

Figure 2 shows the simplified setup without the mannequin. In this case, a sealed volume is created using a syringe. The plunger creates an airtight seal using epoxy or similar. The plunger then uses a leur lock 3 way stopcock to connect to the insufflator and pressure release valve. The flexible tubing is then adapted down to 1/8" ID so it can be connected to the manometer. In this case, a Christmas tree to leur lock adapter is connected to a leur lock to 1/8" barb adapter.

If you encounter an issue that is not described above and prevents the simulator from working properly, please contact us in the GitHub Discussion forums.